TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA
Macroarea di Ingegneria
Dipartimento di Ingegneria Civile e Ingegneria Informatica

Communication in Distributed Systems: RPC

Corso di Sistemi Distribuiti e Cloud Computing

A.A. 2025/26

Valeria Cardellini

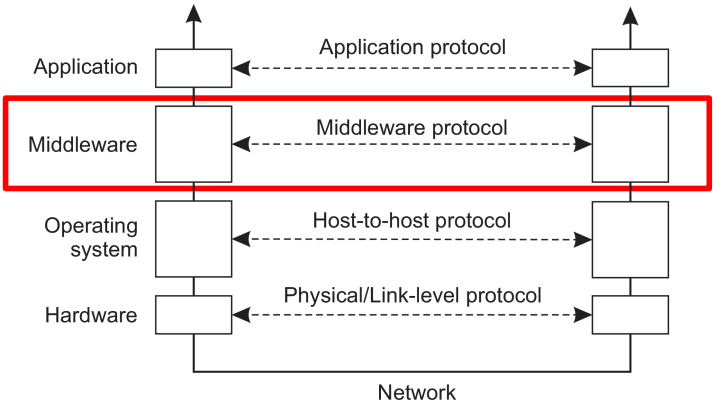
Laurea Magistrale in Ingegneria Informatica

Communication in distributed systems

- Message passing: the basics
- Communication is based on sending and receiving messages
- To make this work, parties must agree on many low-level communication details, such as:
  - How many volts represent a 0 or 1 bit?
  - How many bits represent an integer?
  - How does the receiver know when the message ends?
  - How can the receiver detect message corruption and what to do then?
- The good news: we don't care about these low-level details ☺

Basic networking model and its adaptation

- We already know the solution: divide network communication in layers
  - The well-known ISO/OSI reference model
  - For distributed systems we don't deal with low-level details: we focus on higher-level abstractions built over the network layer
- Adapted layering scheme in distributed systems



Middleware layer

- **Middleware** provides common services and protocols that support different distributed applications and systems
  - General-purpose
  - Application-independent
- Common middleware services
  - Communication: remote procedures calls, message queues, multicasting
  - Naming: identifies and enables sharing of resources
  - Security: ensures secure communication among distributed nodes
  - Distributed consensus: coordinates agreement across multiple distributed nodes
  - Distributed locking: prevents concurrent access to shared resources (e.g., multiple clients editing a file)
  - Data consistency: maintains data consistency across distributed nodes

Communication types in DS

- Let's distinguish communication types by 3 key aspects:

1. Persistency
   - Transient vs. persistent communication
2. Synchronization
   - Synchronous vs. asynchronous communication
3. Time dependence
   - Discrete vs. streaming communication

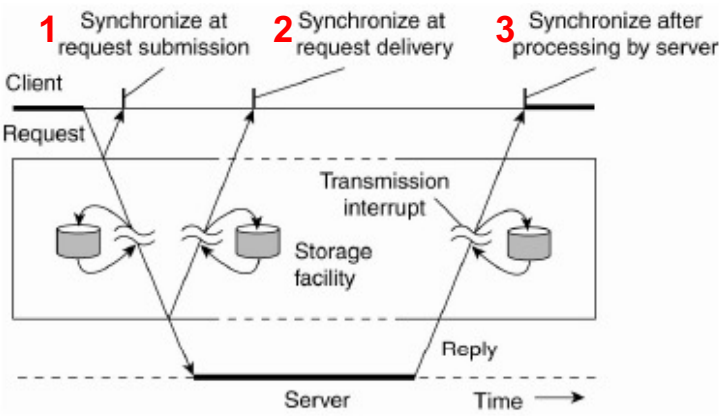
Persistent vs. transient communication

- Persistent communication
  - Message is stored by middleware until it can be delivered
  - Sender can continue execution after submitting the message
  - Receiver does not need to be active at the time of submission

- Transient communication
  - Message is kept only while sender and receiver are both active
  - If delivery is not possible, message is discarded
  - Requires both parties to be executing at the time of communication
  - Transport-layer example: routers store and forward messages, but discard them if forwarding fails

Synchronous communication…

- Once a message is submitted, the sender is blocked until the operation completes
- Send and receive are *blocking* operations
- Synchronization can occur:
  1. At request submission
  2. At request delivery
  3. After request processing



… vs asynchronous communication

- *Non-blocking send*
  - Once the message is submitted, the sender continues processing
  - The message is temporarily stored by middleware until it is transmitted
- *Receive* can be *blocking or non-blocking*
  - Non-blocking receive: immediately returns, either with data (if available) or with an indication that there is nothing to receive yet
- Benefits: decouples sender and receiver, enabling higher concurrency and responsiveness

Discrete vs. streaming communication

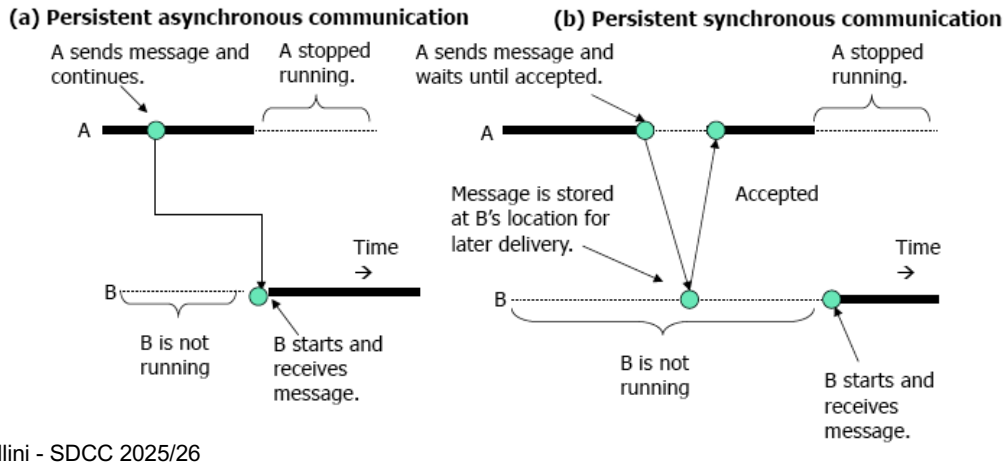
- Discrete communication
  - Each message forms a complete unit of information

- Streaming communication
  - Involves sending multiple messages, in temporal relationship or related to each other by sending order, which is needed to reconstruct complete information
  - E.g., audio, video

Combining persistence and synchronization

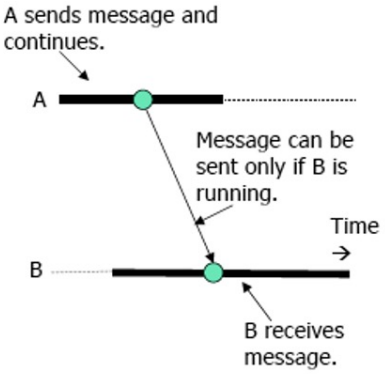
a) Persistent asynchronous communication
  - Message is stored until receiver is ready, sender does not wait for receiver
  - E.g., email, Teams chat, message-oriented middleware

b) Persistent synchronous communication
  - Sender is blocked until message is delivered to receiver
  - Message is stored until delivery is confirmed



Combining persistence and synchronization

c) Transient asynchronous communication
- Sender does not wait
- Message is not stored
- If the receiver is offline or unreachable, message is lost (e.g., UDP)

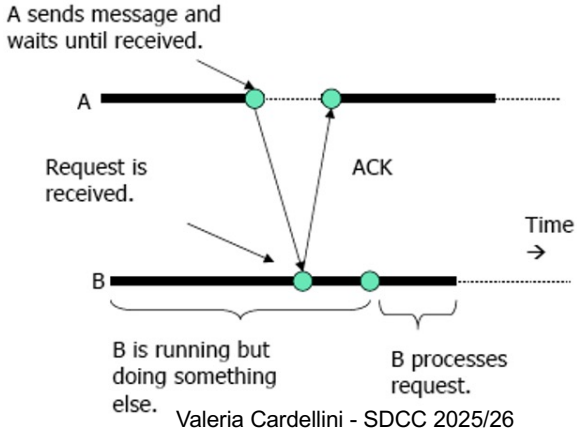


(c)Asynchronous communication

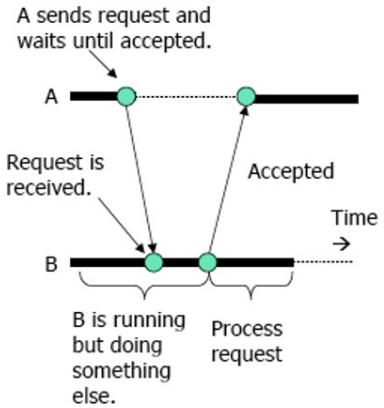
Combining persistence and synchronization

- Message is not stored
- Sender is blocked until a specific condition is met, which alternatives?
  - d) Receipt-based synchronous: sender is blocked until message reaches the receiver buffer (e.g., asynchronous RPC)
  - e) Delivery-based synchronous: sender is blocked until message is actually delivered to receiver

(d) Receipt-based synchronous communication

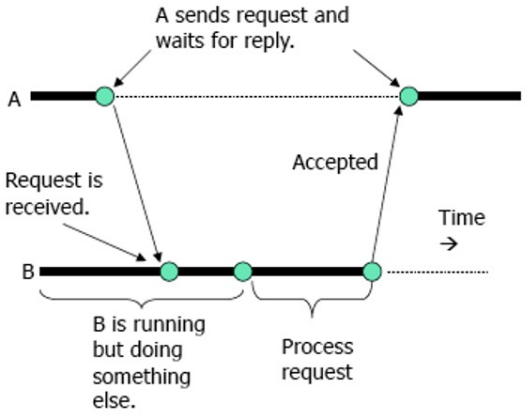


(e) Delivery-based synchronous communication



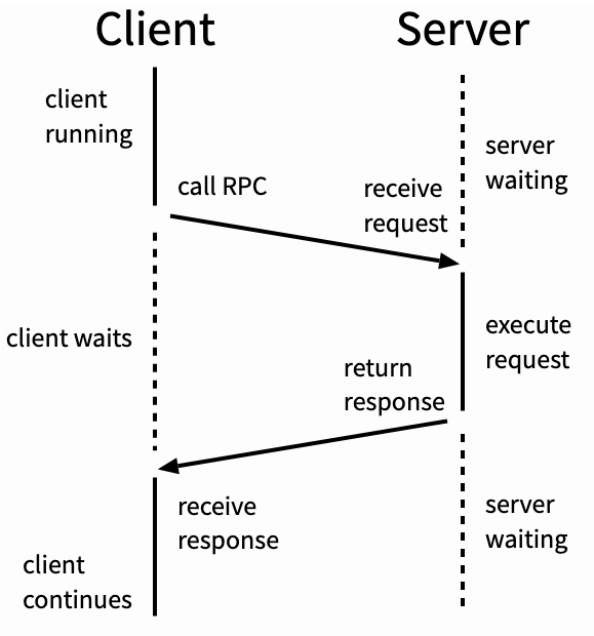
Combining persistence and synchronization

f) Response-based synchronous: sender is blocked until it receives a reply from receiver (e.g., synchronous RPC)



(f) Response-based synchronous communication

What happens when a client calls a RPC

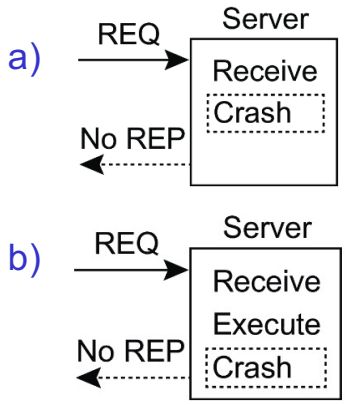


- Which failures can happen?

Communication failures

- Different communication failures between sender (client) and receiver (server): what can go wrong?

1. Message loss or delay
   - Request or reply is lost or delayed
   - Connection is reset
   - Pitfall: *Network is reliable*
2. Server crashes
   a) Before performing the service
   b) After performing the service
   - Client can't distinguish between a) and b)
3. Client crashes after sending request



Failure semantics during communication

- In a DS, communication semantics define what guarantees the DS gives about message delivery in the presence of failures
  - **May-be** semantics
  - **At-least-once** semantics
  - **At-most-once** semantics
  - **Exactly-once** semantics

More guarantees, more complexity

- Failure semantics applies both to message delivery (e.g., MOM) and service processing (e.g., RPC)
- Let's focus on service processing
  - sender = client, receiver = server

Failure semantics in a nutshell

- Maybe: no guarantees that service has been executed or not by server
- At-least-once: service is executed at least once (i.e., >= 1)
  - If client does not receive the reply: service *might* have been executed or it might not, or it might have been executed mutiple times
- At-most-once: service is executed at most once (i.e., 0 or 1)
  - If client receives the reply, it has been processed by server *no more than once*
  - The service may not be executed at all
- Exactly-once: service is executed precisely once
  - Even in the face of retries, crashes, or delays
  - Ideal for critical operations, but complex and costly

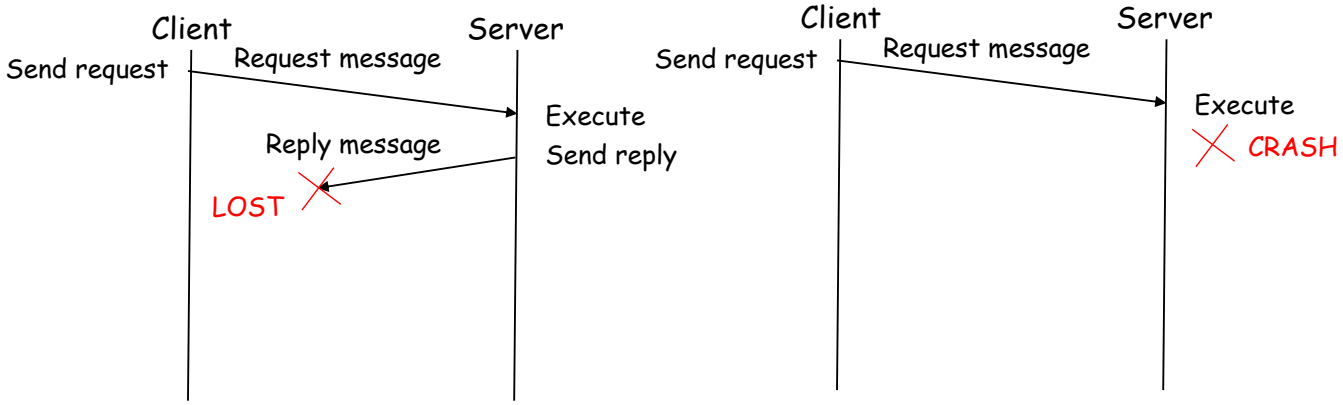
Basic mechanisms for implementation

- 3 basic mechanisms to implement failure semantics

1. Client side: Request Retry (RR1)
   - Client keeps trying until it gets a reply or is confident about server failure after a certain number of failed retries
2. Server side: Duplicate Filtering (DF)
   - Server discards any duplicate request from same client
3. Server side: Result Retransmit (RR2)
   - Server keeps result (reply) so that it can be retransmitted without being computed again when server receives duplicate requests

May-be semantics

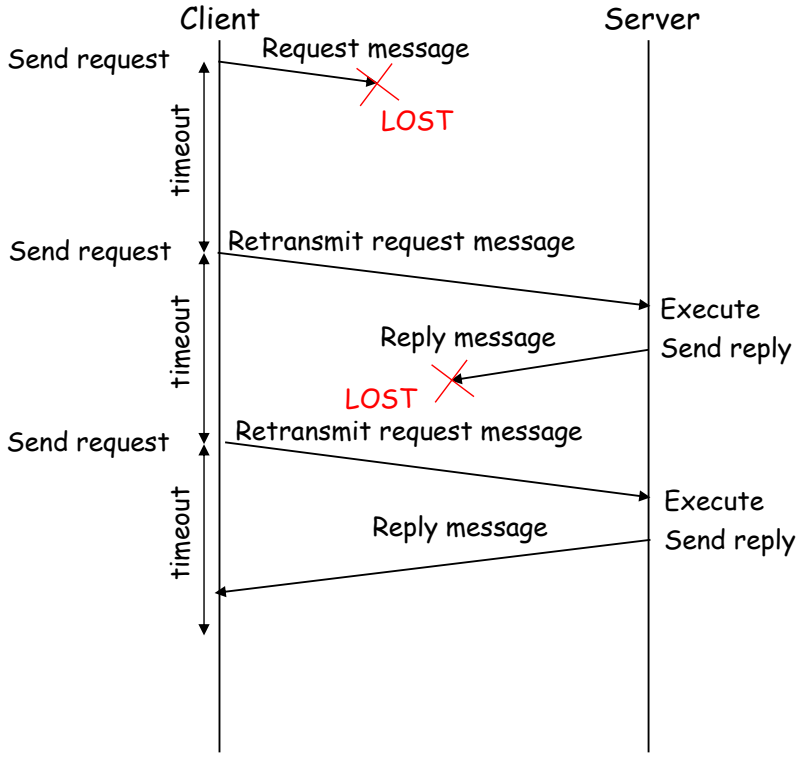
- No guarantee that service has been executed or not by server
- No action is taken to ensure reliable communication: no mechanism (among RR1, DF, RR2) is used
- E.g., best-effort in UDP
- Use cases: logging, sensor data (if occasional loss is tolerable)



At-least-once semantics

- Service, if executed, has been executed at least once
  - Perhaps more than once due to request duplicates caused by retransmissions
- Client uses RR1, server uses neither DF nor RR2
- Upon receiving the reply, client cannot know how many times its request has been processed by server
  - If server executes the service once or multiple times because the reply was lost or delayed
  - If server executed the service but crashed before replying and then recovered, and client retries the request after timeout; server processes the request again
- It works for idempotent services
  - **Idempotent** service: a service where applying it multiple times has the same effect as applying it once to the same input
    - E.g., `get(key)` or `put(key,value)` in KV data store

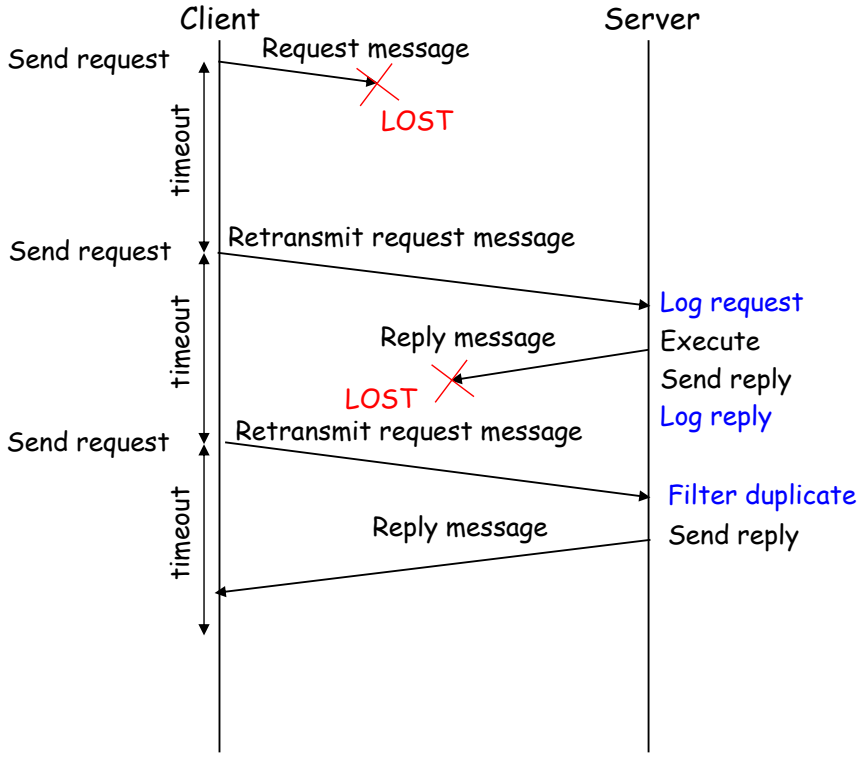
At-least-once semantics



At-most-once semantics

- Service, if executed, has been executed at most once
  - *If* client receives the reply, it knows that the service was executed *once*
  - In case of failure, client cannot determine whether the request never reached the server, or server executed the service but the reply was lost (at-most-once: at most once, but possibly not all)
- Mechanisms used: RR1, DF, and RR2
  - Client retransmits request when timeout expires
  - Server maintains some state to identify duplicate requests and avoid processing the same request more than once
- It works also for non-idempotent services
  - E.g., `increment(counter)` or `withdraw(amount)`
- No strict coordination between client and server
  - Client does not know if the service was executed (in case of no reply), server does not know if client received the reply

At-most-once semantics



At-most-once: implementation

- Server must:
  - Detect duplicate requests
  - Manage duplicate requests returning saved reply instead of re-running service `handler()`
  - Discard saved replies when it is safe

Server:

```
if seen[xid]
 reply = old[xid]
else
  reply = handler()
  old[xid] = reply
  seen[xid] = true
```

- How to detect duplicates
  - Client includes a unique ID (`xid`) with each request and uses same `xid` when retransmitting the request due to timeout
  - Server uses `xid` to detect duplicates
  - If server knows the request with that `xid`
    - If reply is saved, resend it
    - If original request is still executing, don't re-run service

At-most-once: implementation

- How to ensure unique `xid`?
  - Globally unique per client (and per server if needed)
  - Common pattern: `xid = client_id + sequence_number`
  - Sequence number: increments with every new request and wraps around only after large gaps
- How to handle duplicate requests during execution?
  - Server maintains also a status flag: `xid → {status, reply}`
  - Status is `pending` if service still executing
  - Status is `completed` if reply cached
  - If duplicate arrives while status is pending, server ignores it or queues it to wait for the original to finish
- Server must eventually discard cached replies: how?
  - Use sliding windows to track recent `xids`
  - Retain reply for a maximum lifetime

Idempotent Receiver pattern
https://martinfowler.com/articles/patterns-of-distributed-systems/idempotent-receiver.html

Exactly-once semantics

- Strongest and hardest guarantee to implement, especially in large-scale DS
- Requires full agreement on interaction between client and server
  - Service is run only once or not run at all: all-or-nothing semantics
    - If everything goes well: service runs only once, duplicates are found
    - If something goes wrong: client or server knows if service has run (once - all) or not (never - nothing)
- Semantics with concordant knowledge of each other's state and without constraint on maximum duration of interaction protocol between client and server
  - No constraint on maximum duration: barely practical in real systems!

Exactly-once semantics: more mechanisms

- Basic mechanisms (RR1, DF, RR2) are not enough
  - Do not protect against server failures
- Additional mechanisms for fault tolerance
  - Transparent server replication
    - Multiple servers replicate the same service
    - E.g., leader-follower or primary-backup
  - Write-ahead logging (WAL)
    - Before executing a service, the server logs the request to persistent storage
  - Recovery mechanisms
    - After failure, the server resumes processing from a safe state
    - *State checkpoint*: saves a local snapshot of server's state to persistent storage
      - Combined with WAL to reduce recovery time (only replay log entries after last checkpoint)
    - *Snapshot*: captures a consistent global state of replicated servers: more complex, requires coordination

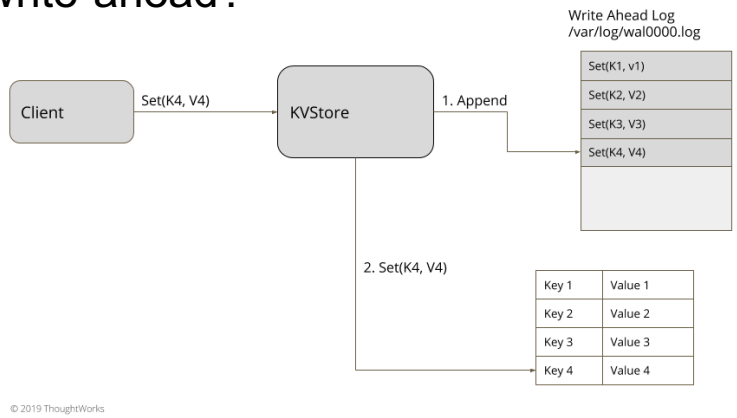
Write-ahead logging pattern

- aka Commit log
- Goal
  - Ensure durability by persisting every state change so server can survive crashes and restarts.
  - Log is stored
- How
  - Each state change is stored as log entry in an append-only commit log file written on persistent storage (e.g., disk or SSD)
  - Entries are appended sequentially, ensuring fast write performance and crash-consistent ordering
  - On server restart, log is read and all entries are replayed in order to rebuild the in-memory state
  - No state is lost, as long as the log was flushed to disk before crash

WAL pattern
https://martinfowler.com/articles/patterns-of-distributed-systems/wal.html

Write-ahead logging pattern

- Why write-ahead?




```
public void put(String key, String value) {
    appendLog(key, value);
    kv.put(key, value);
}

private Long appendLog(String key, String value) {
    return wal.writeEntry(new SetValueCommand(key, value).serialize());
}
```


Summing up failure semantics

- In practice, at-least once and at-most once semantics are both feasible and widely used in DS
- Exactly-once is ideal, but often too complex, slow, or expensive to achieve in practice

- We often choose the lesser of two evils, i.e., at-least-once semantics
  - Easier to scale
    - Stateless retries
    - No need for coordination between sender and receiver
  - Works well with idempotent services
  - Can be combined with application-level deduplication if needed

Distributed systems are all about trade-offs!

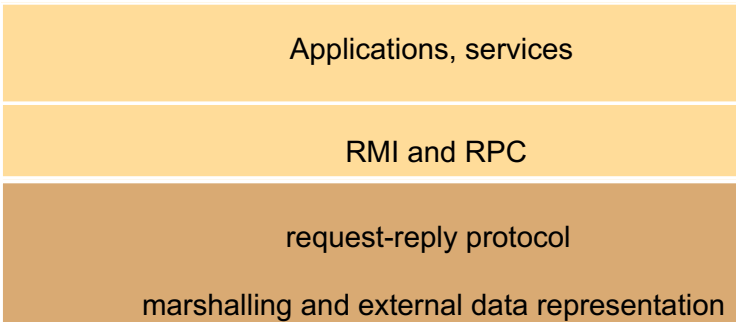
Distributed application programming

- Explicit network programming
  - Operating system constructs based on socket API and explicit management of message exchange
  - Foundational for network applications (e.g. web browser, web server, chat app)
  - ✗ Distribution is not transparent and requires developer effort

- How to increase the abstraction level?
- Communication middleware between OS and applications
  - Provides a higher-level API for communication
  - Hides complexity of underlying hw and sw layers
  - Frees developers from automatable tasks
  - Improves software quality by providing reusable, correct, and efficient solutions to common DS challenges

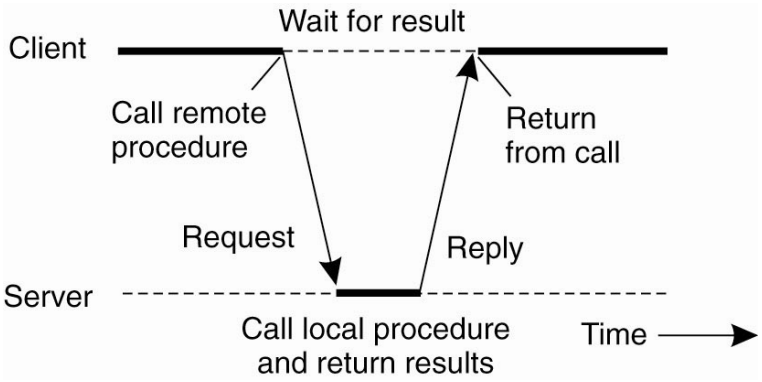
Distributed application programming

- Implicit network programming
  - Language-level constructs simplify distributed programming by hiding network details from developers
  - Developers work with **remote procedure calls (RPC)** or **remote method invocations (RMI)** as if they were local calls
  - In reality, the caller (client) and callee (server) are located on separate machines



Remote Procedure Call (RPC)

- Core idea (Birrel and Nelson, 1984): use client/server model to call procedures executed on remote machines
  - Client on machine A calls a procedure on machine B
  - Calling process on A is suspended
  - Called procedure is execute on B
  - Input/output parameters are transported as messages
  - No message passing is visible to developer



Why RPC

- Widely used in many distributed systems, including cloud computing, microservice architectures
- Many languages and frameworks have they own implementation of RPC:
  - **C (Sun RCP)**
  - **Java (Java RMI)**
  - **Python (RPyC)**
  - **Go**
  - **gRPC**

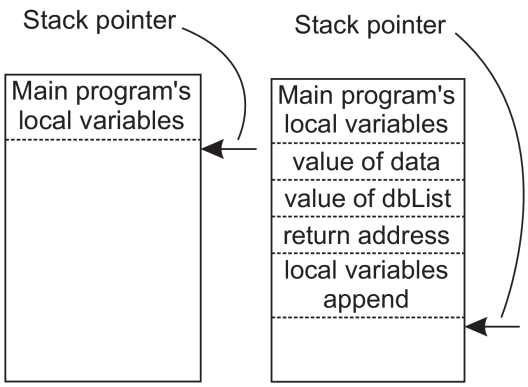
  Our case studies
  - Distributed Ruby https://github.com/ruby/drb
  - Ice https://zeroc.com/ice
  - Microsoft .NET https://dotnet.microsoft.com/
  - JSON-RPC https://www.jsonrpc.org

Local procedure call

- Example of local procedure call:

```
newlist = append(data, dbList)
```

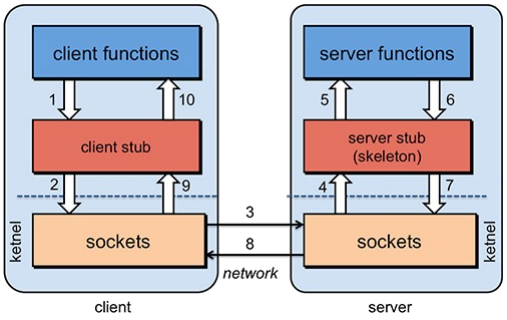
- Caller pushes to stack input parameters (`data`, `dbList`) and return address
- When callee returns, control is back to caller



How can RPC appear as close as possible to a local procedure call?

RPC: architecture

- Solution: proxies (aka **stubs**) to make RPC calls feel like local calls
- Client side: **client stub** exposes service's interface
  - Client calls client stub, which manages all communication details
- Server side: **server stub** receives request and dispatches it to the local procedure on the server
- Goal: distribution transparency
  - Automatic stub generation
  - Focus on application logic

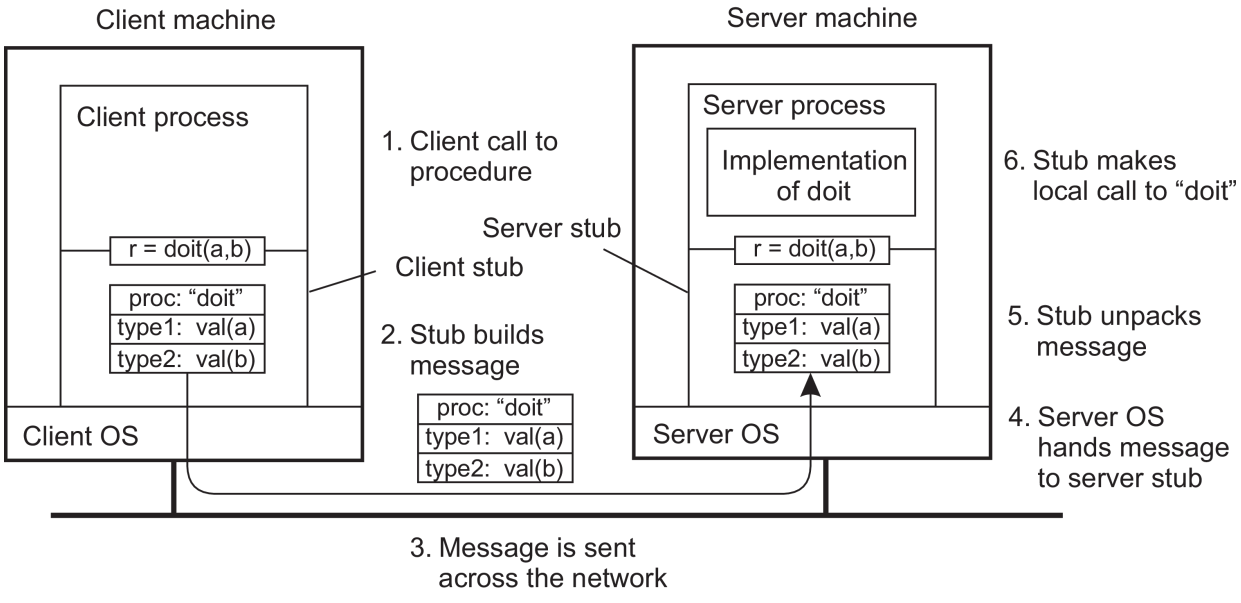


RPC: basic steps

1. On client side, client calls a local procedure, the client stub
2. Client stub marshals parameters (converts them to a common format), packages them into a request message, and call local OS
3. Client OS sends request message to remote OS
4. On server side, remote OS delivers request message to server stub
5. Server stub unmarshals parameters and calls server as it were a local procedure
6. Server executes local call and returns result to server stub
7. Server stubs marshals return value(s) and calls OS
8. Server OS sends reply message to client OS
9. On client side, Client OS delivers reply message to client stub
10. Client stub unmarshals return value(s) and returns result to client

RPC: example

- RPC `doit(a,b)`



RPC: key requirements

- To make an RPC appear as local procedure call, the middleware must:
- Exchange messages between client and server
  - Identify request and reply messages, and remote procedure
  - Pass parameters and return values
- Manage data heterogeneity
  - Parameters, return value(s)
  - Marshaling vs. serialization:
    - ***Marshaling***: bundle parameters into a transferable form that can be unmarshaled (reconstructed) by another process
    - ***Serialization***: convert objects into a sequence of bytes for transmission; serialization is a subset of marshaling
- Handle failures
  - During communication (e.g., message loss, delay)
  - User errors (e.g., invalid input)

RPC: implementation challenges

1. Manage heterogeneity in data representation
   - In addition, client and server must agree on transport protocol for message passing: TPC, UDP, both?
2. Perform parameter passing by reference
   - Client and server run on different machines with separate address space
3. Define failure semantics
   - Local procedure call: exactly-once
   - Remote procedure call: typically at-least-once or at-most-once
4. Bind client to server
   - Locate server endpoint before performing the call

Data representation heterogeneity

- Client and server may use different data representations
  - E.g., byte ordering (little-endian vs big-endian), data size, padding, …
  - RPC must define how RPC messages are encoded on the wire

- Alternatives to handle data representation heterogeneity (*general*, not only RPC):
  1. Specify encoding within the message itself
  2. Sender converts data into receiver encoding
  3. Use a common encoding agreed by both parties
     - Sender converts from local to common format
     - Receiver converts from common to local format
  4. Use an intermediary to convert between different encodings

Data representation heterogeneity

- Comparing alternatives for data conversion, assuming *N* components
- #2: each component must know conversion functions for every other component
  - Number of conversion functions: *N**(*N*-1)
  - ✓ Faster conversion (direct)
  - ✗ Poor scalability
- #3: all components agree on common encoding, each component only needs to convert to and from local format
  - Number of conversion functions: 2*(*N*-1)
  - ✗ Slower conversion (2 steps)
  - ✓ Better scalability
- #3: standard choice in most RPC systems

Data representation heterogeneity: patterns

- Patterns to implement alternatives #3 and #4
- Proxy (aka stub)
  - Goal: support access and location transparency
  - Manage access to a remote object via a proxy
    - Proxy resides in local address space and exposes the same interface of remote object
- Broker
  - Goal: separate and encapsulate communication details from its functionality
  - Enable components to interact without handling remote concerns themselves
  - Handles tasks such as server location, message routing, and communication hiding
- Proxy: standard choice in RPC systems
  - Who automatically generates stubs?

Parameter passing methods

- Call by value
  - Parameter value is copied into local storage (usually stack)
  - Callee acts on copy; changes do not affect caller data
- Call by reference
  - Reference (pointer) to parameter is passed (copied into stack)
  - Callee acts directly on caller data
- Call by copy-restore
  - Special case of call by reference
  - Data is copied into callee stack; upon procedure return, updated data is copied back (restored) to caller
  - Supported in a few programming languages (e.g., Ada, Fortran)

RPC parameter passing

- A reference is a memory address
  - Valid only in its context (local machine)
  - We need a pointer-less representation

- Solution: simulate call by reference in RPC by using call by copy-restore
  - Client stub copies the pointed data into the request message
  - Server stub works on copy in its own address space
  - If modified, the updated copy is restored by client stub overwriting original data
  - Size of data to be copied must be known
  - What happens if data contains pointers?

RPC failure semantics

- Exactly-once semantics
  - Guarantees remote procedure is executed exactly once
  - Costly and complex to implement
- At-least-once semantics
  - If client receives a reply, the remote call has been executed at least once
  - Possible duplicate executions due to retransmissions
  - Suitable for idempotent operations
- At-most-once semantics
  - If client receives a reply, the remote call has been executed at most once
  - No duplicate executions, but call may not have been executed at all in failure scenarios
  - Requires duplicate request detection and state management on server

Binding: locating the server endpoint

- Binding: find the server endpoint, including address and specific process (e.g., port)
  - Types of binding: static or dynamic
- Static binding
  - Server endpoint known at design time (hard-coded)
  - ✓ Simple, no runtime overhead
  - ✗ Lacks transparency and flexibility
    - Changes require redeployment
- Dynamic binding
  - Server endpoint determined at run-time
  - ✓ Gains transparency and flexibility
    - Request redirection in case of server replication
  - ✗ Increased overhead: how to limit it?

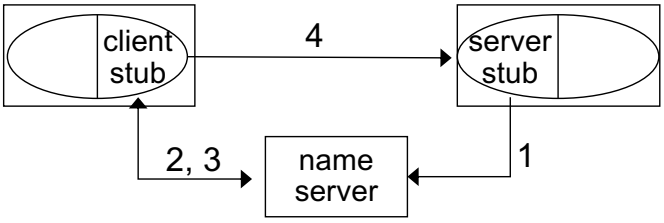
Dynamic binding

- Two phases in client/server relationship

1. Naming: static phase *before* execution
   - Client specifies whom it wants to connect to, using a unique service name
   - Unique names correspond to operations or abstract interfaces
   - Binding is made to the specific service interface, not a concrete address
2. Addressing: dynamic phase *during* execution
   - Actual server endpoint are resolved when client invokes the service
   - Middleware may dynamically select among multiple replica servers

Dynamic binding

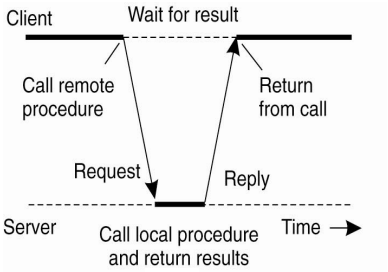
- Addressing can be explicit or implicit
  - Explicit addressing: client sends request using broadcast or multicast, waiting for the first reply
  - Implicit addressing: uses a name server (aka binder, directory service, registry service) that registers services and manages a binding table
    - Supports service lookup, registration, update, and deletion



- Addressing frequency
  - Each RPC requires addressing
  - To reduce overhead, binding results are cached and re-used

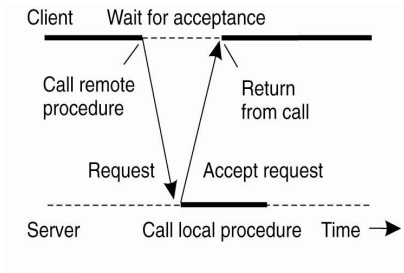
Other issues: Synchronous vs asynchronous RPC

- Synchronous RPC: strict request-reply behavior
  - RPC call blocks client that waits for server reply



Synchronous RPC

- Some RPC middleware supports asynchronous RPC
  - Client does not wait for server reply: continues execution
  - Server can send an ack upon receiving the request is received and execute procedure later



Asynchronous RPC

Other issues: transparency

- Is RPC truly transparent? Can RPCs be treated like local calls?
  - Performance, failures, concurrent requests, replication, migration, …
- Performance
  - RPC is much slower than local calls: why?
  - Local procedure call: ~10 cycles (~3 ns)
  - RPC on LAN: 0.1-1 ms => ~100000 times slower
    - Major overheads: context switching, data copying, communication
  - RPC over WAN: can be millions of times slower

Other issues: transparency

- Common failures in RPC
  - Client cannot locate server
  - Lost request messages during transmission
  - Server crashes during execution
  - Lost reply messages
  - Client crashes before or after request

Other issues: security

- Authentication
  - Does client send messages to genuine server or to impostor?
  - Is the client who it claims to be?
- Authorization and access control
  - Does server accept messages only from legitimate clients?
  - Can server identify users at client side?
- Messages confidentiality and integrity
  - Messages may be visible and sniffed on network
  - Can messages be encrypted?
  - Are messages protected against accidental corruption or truncation while on network?
- Replay attacks
  - Can a malicious host capture and retransmit a message later?

Programming with RPC

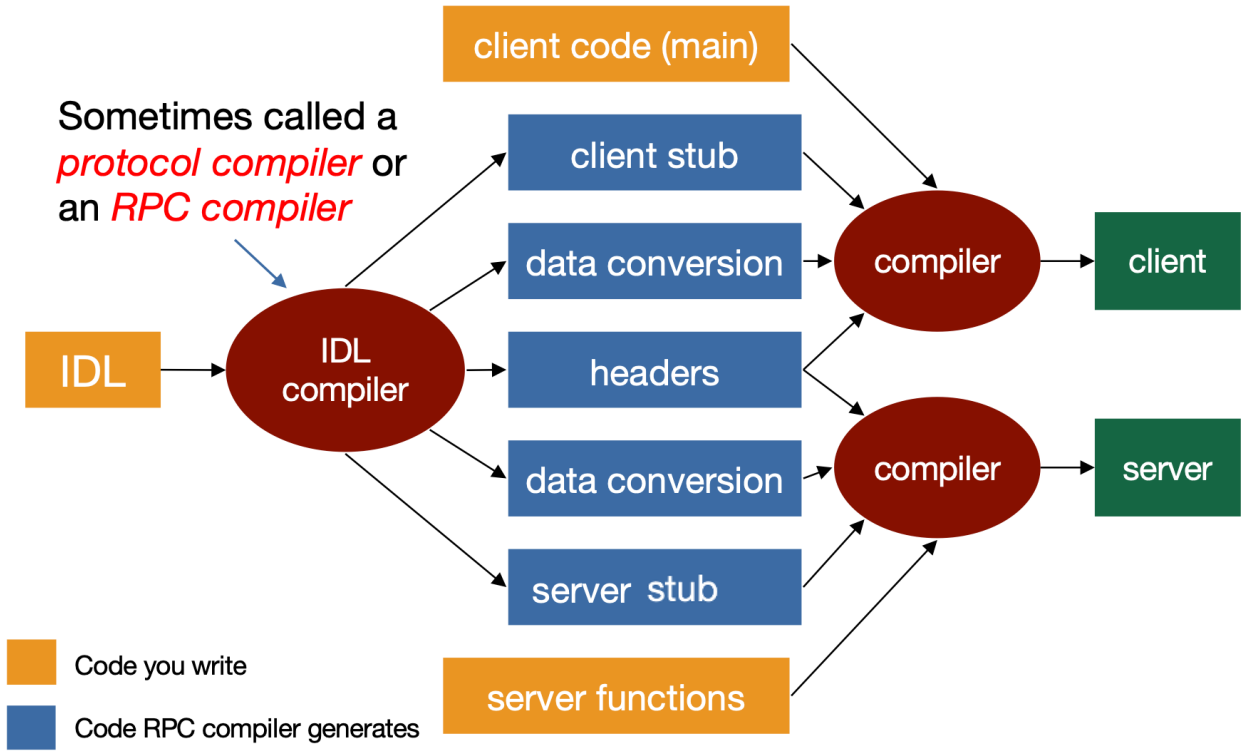
- Language-level support
  - Some languages (e.g., C, C++) lack built-in RPC support
  - Others (Java, Python, Go, Erlang, Haskell) provide direct support
    - Challenge: heterogeneous environments (e.g., Java service communicating with Python service)
- Common solution
  - Use an **Interface Definition Language (IDL)** to describe remote procedures
  - A dedicated IDL compiler automatically generates client and server stubs

Interface Definition Language (IDL)

- IDL allows developers to specify remote procedure interfaces (including *names, parameters, return values*) in a machine-independent way
- IDL compiler uses these interfaces to generate client and server stubs, containing:
  - Code to marshal
  - Code to unmarshal
  - Network transport code
- IDL definitions look similar to function prototypes
- Example: Thrift IDL

```
service KeyValueStore {
  i32 put(1: string key, 2: string value),
  string get(1: string key),
}
```


IDL and its compiler



RPC middleware: case studies

- Sun RPC
- Java RMI
- Python RPyC
- Go
- gRPC

RPC implementation: Sun RPC

- First-generation RPC
- Created by Sun Microsystems (now Oracle)
  - RFC 1831 (1995), RFC 5531 (2009)
  - Still used mainly for NFS (Network File System)
- Interfaces defined using XDR (eXternal Data Representation) as IDL
  - IDL compiler: `rpcgen`
  - Provides built-in conversion functions for:
    - Primitive types, e.g., `xdr_int()`
    - Structured types, e.g., `xdr_string()`
  - Binary format with *implicit typing*
    - Only values sent, no type or parameter metadata

Sun RPC: define RPC program using XDR

- Two main parts in a file `.x`
  1. Definition: procedure specifics to identify procedure names and parameters' data types
  2. XDR definitions: custom data definitions (if not built-in)

- Example: procedure to calculate the square of an integer

Sun RPC example: define remote procedure

`square.x`

```
struct square_in {       /* input (argument) */
  long arg1;
};
struct square_out {      /* output (result) */
  long res1;
};
program SQUARE_PROG {
  version SQUARE_VERS {
  square_out  SQUAREPROC(square_in) = 1; /* procedure number = 1 */
  } = 1;                 /* version number */
} = 0x31230000;          /* program number */
```

- Define remote procedure `SQUAREPROC`
  - Each procedure has only one input parameter and one output parameter
  - Identifiers are written in uppercase
  - Each procedure is associated with a unique procedure number within RPC program (e.g., 1)

Sun RPC: how to implement RPC program

- Developer responsibilities in RPC
- Client program: implements `main()` and logic needed to locate and bind remote procedure (`square_client.c`)
- Server program: implements remote procedures provided by RPC server (`square_server.c`)
  - Note: developer does not write server-side `main()`
  - Who calls the remote procedure on server side?

Example: local procedure

- Let's first consider the standard local procedure

`square_local.c`

```
#include <stdio.h>
#include <stdlib.h>

struct square_in {  /* input (argument) */
  long arg;
};
struct square_out {  /* output (result) */
  long res;
};
typedef struct square_in square_in;
typedef struct square_out square_out;

square_out *squareproc(square_in *inp) {
  static square_out out;

  out.res = inp->arg * inp->arg;
  return(&out);
}
```

Example: local procedure

- Local procedure *(continue)*

```
int main(int argc, char **argv) {
  square_in in;
  square_out *outp;

  if (argc != 2) {
    printf("usage: %s <integer-value>\n", argv[0]);
    exit(1);
  }
  in.arg = atol(argv[1]);

  outp = squareproc(&in);
  printf("result: %ld\n", outp->res);
  exit(0);
}
```

- Remote procedure: what changes?

Sun RPC example: remote procedure

- Remote vs. local: *similar*, not the same

server.c

```
#include <stdio.h>
#include <rpc/rpc.h>
#include "square.h"    /* generated by rpcgen */
square_out *squareproc_1_svc(square_in *inp, struct svc_req
   *rqstp) {
  static square_out out;

  out.res1 = inp->arg1 * inp->arg1;
  return(&out);
}
```

- Input and output parameters use pointers
- Output must point to a static variable so it persists after procedure returns
- Procedure naming convention: original RPC procedure name becomes `<procedure_name>_<version>_svc`, e.g., `squareproc_1_svc`, all lowercase)

Sun RPC example: client

- Run RPC client:

  `./square_client <server-hostname> <integer>`

client.c

```
#include <stdio.h>
#include <rpc/rpc.h>
#include "square.h"   /* generated by rpcgen */
int main(int argc, char **argv) {
  CLIENT *clnt;
  char *host;
  square_in in;
  square_out *result;
   if (argc != 3) {
    printf("usage: client <hostname> <integer-value>\n");
    exit(1);
  }
  host = argv[1];
  clnt = clnt_create(host, SQUARE_PROG, SQUARE_VERS, "tcp");
```

```
CLIENT *clnt_create(char *host, unsigned long prog,
unsigned number vers, char *proto)
```


Sun RPC example: client

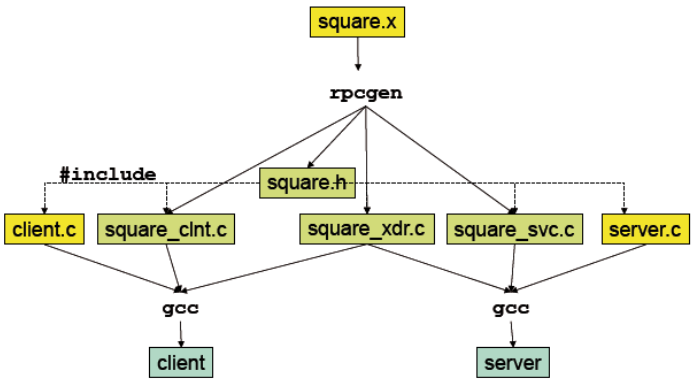
```
  if (clnt == NULL) {
    clnt_pcreateerror(host);
    exit(1);
  }
  in.arg1 = atol(argv[2]);
  if ((result = squareproc_1(&in, clnt)) == NULL) {
    printf("%s", clnt_sperror(clnt, argv[1]));
    exit(1);
  }
  printf("result: %ld\n", result->res1);
  exit(0);
}
```

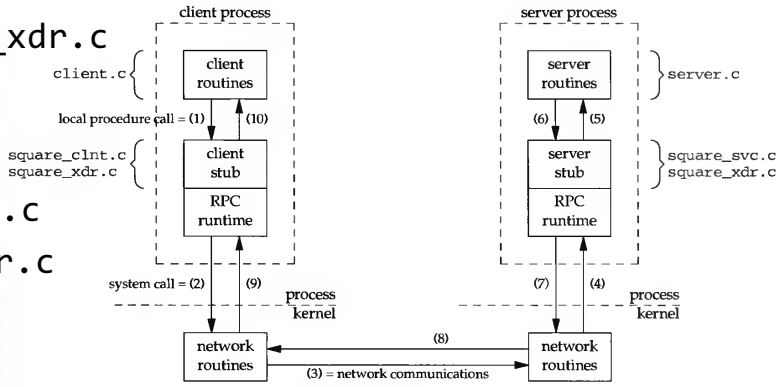
Sun RPC example: client

- Creation of client transport manager: `clnt_create()`
  - Handles communication (TPC or UDP)
  - Default request timeout and retransmission
- Client must know:
  - Server hostname
  - Remote procedure info: program (`SQUARE_PROG`), version (`1`), procedure (`squareproc`)
- To call the remote procedure:
  - Procedure name: lowercase + _ + version (`squareproc_1`)
  - Input: actual parameter + client transport manager
  - Output: pointer to result
  - RPC failure: function returns `NULL`
- Error handling:
  - `clnt_pcreateerror()` and `clnt_perror()`

SUN RPC example: development process

1. Define service specification
   - Using XDR: `square.x`
2. Use `rpcgen` to generate stubs
   - Header: `square.h`
   - Client stub: `square_clnt.c`
   - Server stub: `square_svc.c`
   - XDR routines: `square_xdr.c`
3. Implement client and server procedures
   - Client program: `client.c`
   - Server program: `server.c`





Sun RPC: server binding

- Client stub needs to know port number: how?
- Server stub registers RPC program
  - Each procedure is identified by program number, procedure number, version number
- Information stored in *port map*
  - Dynamic table of RPC services on server host
  - Managed by *port mapper* (`rpcbind`), listening on port 111
  - Client stub contacts port mapper to discover the port number, then sends the request to server stub

Sun RPC: port mapper (rpcbind)

- List RPC programs

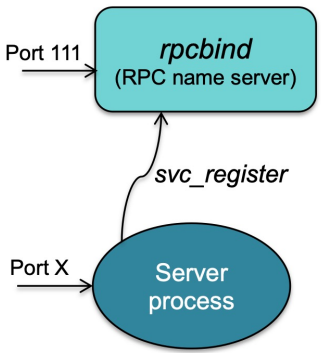
```
>$ rpcinfo -p
 program         vers     proto    port
   100000        4        tcp      111       rpcbind
   100000        4        udp      111       rpcbind
   824377344     1        udp      59528
   824377344     1        tcp      49311
```

Our RPC program `SQUAREPROC`

- 824377344 (= 0x31230000) is the program number in square.x
- Server supports both TCP and UDP: transport-protocol independent

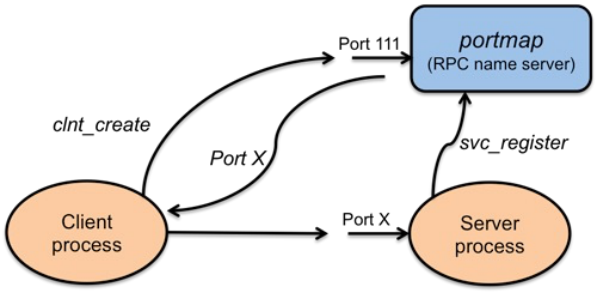
What goes on: server side

- Server stub code (`square_svc.c`)
- In `main()`:
  - Creates a socket and binds it to any available local port
  - Calls `svc_register` (RPC library function)
    - Registers procedures with port mapper (port 111)
    - Associates the specified program and version number pair with the dispatch routine
  - Calls `svc_run` (RPC library function)
    - `svc_run` enters the server's main loop
    - Listens for RPC requests
    - Dispatches to correct service procedure based on program and procedure numbers
    - Sends RPC replies



What goes on: client side

- When client program starts, `clnt_create` contacts server's port mapper to get port for that RPC interface
  - Early binding: done once, not per call



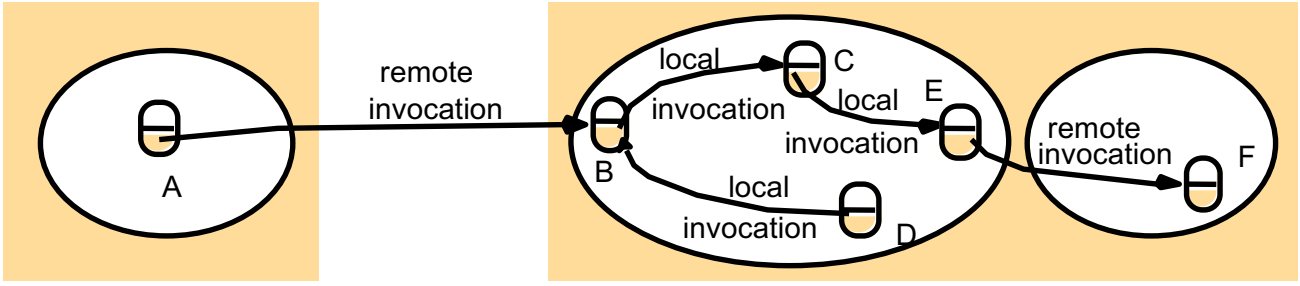
- Client stub (`square_clnt.c`) manages communication
  - Handles request timeout
  - Marshals/unmarshals between local and XDR format

Sun RPC: summing up key features

- Remote procedures
  - RPC program can expose multiple procedures
  - Versioning support
  - One input and one output per procedure
  - Call by copy-restore
- Transport independent: TCP or UDP at run-time
- Synchronous, blocking calls
- Concurrency: default is no concurrency, mutual exclusion guaranteed
- Stub code: visible, auto-generated by `rpcgen`
- Failure semantics: at-least-once
  - Retransmission on timeout
- Server binding: port mapper
- Security: authentication added with Secure RPC

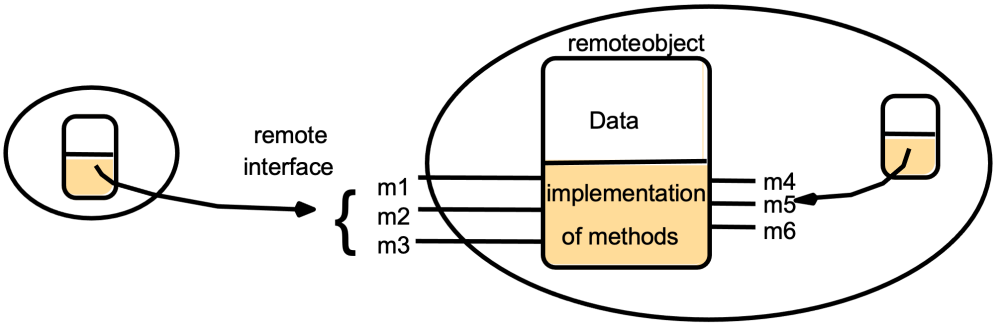
Java RMI

- Java RMI (Remote Method Invocation)
- Second-generation RPC
- RPC for Java, extended to distributed objects
  - Objects on one JVM can invoke methods on objects in another JVM
  - Goal: access transparency
  - Distribution transparency: not full



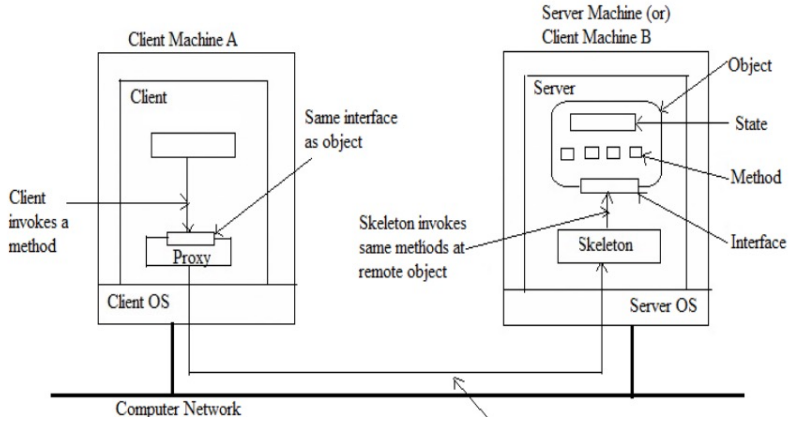
RMI: basics

- Physical separation possible due to logical separation
  - Java separation between definition (interface) and implementation (class)
- Remote interface: defines methods available for remote invocation
- Remote object: instance of class implementing remote interface
  - Important: internal state of remote object is not distributed



RMI: basics

- Remote method invocation: invoke methods of a remote interface on a remote object
  - Goal: keep same syntax as local method call
- Achieved via proxy pattern
  - Client-side **stub**: represents remote object locally
  - Server-side **skeleton**: dispatches calls to actual object

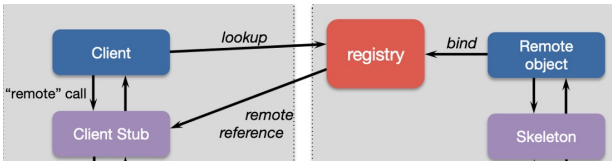


RMI: serialization/deserialization

- No external data representation needed
- Serialization/deserialization handled by Java
- Serialization: converts object passed as parameter to byte stream (`writeObject`)
- Deserialization: decodes byte stream to object (`readObject`)
- Stub and skeleton use this to exchange parameters and return values between JVMs
- Marshaling vs serialization
  - Marshaling: convert local data into network format for transmission
  - Serialization: convert object state into byte stream, reconstructs object at receiver
  - Note: Java distinguishes them (needs codebase), Python Pickle treats them as the same

RMI: call flow

- Client obtains object remote reference (stub) via **RMI registry**
  - Name server that relates objects with names
  - Server registers remote objects with registry
  - Client looks them up by name



- Client invokes remote method on stub
  - Syntax same as local call
- Stub serializes method ID and parameters and sends them to skeleton
- Skeleton receives message, deserializes data, invokes method, serializes return value, and sends it to stub
- Stub receives message, deserializes return value, and returns it to client

RMI example: echo remote interface

- Remote interface extends **`Remote`**
  - **`Remote`** identifies interfaces whose methods may be invoked from non-local JVMs
- Remote method
  - throws **`RemoteException`**
    - To handle failures (network, protocol, server)
    - Remote invocation is not fully transparent: failures are visible
  - passes parameters
    - by value: primitive types or objects implementing `Serializable`; stub serializes, skeleton deserializes
    - by reference: remote objects
      - Reference is a data structure containing {IP address, port, time, object ID, remote interface}, not a pointer

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface EchoInterface
    extends Remote{
 String getEcho(String echo)
    throws RemoteException;
}
```


RMI example: echo server

- Class implements remote interface
  - Extend **`UnicastRemoteObject`**
  - Call class constructor **`UnicastRemoteObject`** which enables server to receive requests
  - Implement remote method, which throws remote exception

```
public class EchoRMIServer
 extends UnicastRemoteObject
 implements EchoInterface {

 // Constructor
 public EchoRMIServer()
  throws RemoteException {
    super();
 }

 // Implement remote method
 public String getEcho(String echo)
   throws RemoteException {
     return echo;
   }
```

RMI example: echo server

- Create instance of remote object
- Start RMI registry
- Register remote object with registry by name
  - Registry and server run on same host

```
public static void main(String[] args) {
  final int REGISTRYPORT = 1099;
  String registryHost = "localhost";
  String serviceName = "EchoService";
  try {
    // Create an instance of echo server
    EchoRMIServer serverRMI = new EchoRMIServer();
    // Bind remote object with RMI registry
    Registry registry =
      LocateRegistry.createRegistry(REGISTRYPORT);
    registry.bind(serviceName, serverRMI);
  }
  catch (Exception e) {
    System.err.println("EchoRMIServer exception: ");
    e.printStackTrace();
  }
 }
}
```


RMI example: echo client

- Lookup remote object in server's RMI registry by name
- Registry returns a remote reference (stub)
- Client invokes remote method
  - Synchronous, blocking

```
public class EchoRMIClient
{
  // Start RMI client
  public static void main(String[] args)
 {
   bufferedReader stdIn =
     new BufferedReader(
       new InputStreamReader(System.in));
  try
  {
    // Connect to remote RMI service
    EchoInterface serverRMI = (EchoInterface)
         Naming.lookup("EchoService");
    // Interact with user
    String message, echo;
    System.out.print("Message? ");
    message = stdIn.readLine();
    // Invoke remote service
    echo = serverRMI.getEcho(message);
    System.out.println("Echo: "+echo+"\n");
  }
  catch (Exception e)
  {e.printStackTrace(); System.exit(1); }
 }
}
```

RMI: summing up key features

- Transparency: remote calls appear local
- Stub and skeleton auto-generated
- Synchronous, blocking calls
- Concurrency: multiple clients can invoke the same remote method
  - Thread-safety required (`synchronized`)
- Failure semantics: at-most-once
- Full support for object serialization/deserialization and dynamic class loading
- Server binding: RMI registry for object lookup
- Exception handling: `RemoteException`
- Security: optional security manager

Python RPC

- Various RPC implementations
  - Pyro https://pyro5.readthedocs.io/
  - RPyC
  - ZeroRPC https://www.zerorpc.io
- What helps Python achieve transparency
  - Runtime introspection: ability to examine objects during execution via `inspect` module https://docs.python.org/3/library/inspect.html
    - Inspect live objects, examine class contents, retrieve method source code, and extract function argument list
- General idea of RPC in Python:
  - Create connection using an RPC object
  - Invoke remote methods using that object
- Case study: RPyC (Remote Python Call) https://rpyc.readthedocs.io

RPyC: features

- Transparent RPC in Python
  - Transparent access to remote objects as if they were local
  - Proxy-based, but RPyC does not use stubs as other RPC middleware
  - Pass objects by value (immutable) or by reference (mutable, via `netref`)
  - Lightweight, pure Python: no interface definition file, IDL compiler, name server, transport service
- Symmetric operations
  - Client and server can call RPCs on each other (e.g., server can invoke callbacks on client)
- Synchronous and asynchronous calls
- Secure
  - Capability-based security model
  - Integrates with TLS/SSL, SSH

RPyC: server and client

- Server
  - Multiple implementations
    - `ThreadedServer`: thread for each connection
    - `OneShotServer`, `ThreadPoolServer`, `ForkingServer`: for different concurrency models
  - Binds to default port (18812, 18821 for SSL)
    - Developer can specify host IP and port
- Client
  - Connects to server
  - Uses `modules` property to access server module namespace
  - Transparent proxy forwards operations to remote objects

RPyC: passing data

- RPyC uses marshaling (serialization) internally, mainly via Python's `pickle` module
- By value
  - Simple or immutable types (`int`, `string`, `tuple`)
  - Values are sent directly to remote side
  - Remote copy is independent of the original
- By reference
  - Mutable or complex objects (e.g., open files)
  - Passed as `netrefs` (network references)
  - Remote server contacts client to access attributes or call methods on the original object
    - Changes affect the original local object
  - Enables location-sensitive objects (files, OS resources, `sys.stdout`)
    - E.g., remote process can print to local console via `sys.stdout`

RPyC: transparent remote access

- Transparent remote access
  - Client creates a dynamic proxy (`netref`) for remote objects when it connects to the server
  - Proxy acts like the real object: all operations are automatically forwarded to the remote side
  - Client is unaware that the object is remote
- Synchronous and asynchronous calls
  - Synchronous: client blocks and waits for return
  - Asynchronous: immediate return, notification when complete
  - Wrap calls with `rpyc.async_` to make them asynchronous

RPyC: services and security

- RPyC is built around services
  - Each connection exposes a service that defines the set of allowed (its policy)
- Service definition
  - Services are classes derived from `rpyc.Service`
  - They define objects and methods that can be accessed remotely (exposed)
- Exposing methods
  - A method can be called remotely when:
    - Its name begins with `exposed_`
    - Or is decorated with `@rpyc.exposed`
  - Non-exposed methods remain private to the server

RPyC: services and security

- Exposing classes
  - A class whose name begins with `exposed_` can be instantiated by the client
  - However, only its exposed methods are accessible remotely
- Key rule: in RPyC, only exposed classes, methods and objects are visible to the remote side
  - Anything not exposed remains private to the server
- Exposure matters for security
  - Whatever you expose defines your remote attack surface

RPyC example: calculator server

```python
import rpyc

class CalculatorService(rpyc.Service):
    def exposed_add(self, a, b):
        return a + b
    def exposed_sub(self, a, b):
        return a - b
    def exposed_mul(self, a, b):
        return a * b
    def exposed_div(self, a, b):
        return a / b
    def exposed_fib(self,n):
        seq = []
        a, b = 0,1
        while a < n:
            seq.append(a)
            a, b = b, a+b
        return seq
    def foo(self):
        print("foo")

if __name__ == "__main__":
    from rpyc.utils.server import ThreadedServer
    t = ThreadedServer(CalculatorService, port=18861)
    print('Service started on port 18861')
    t.start()
```


RPyC example: calculator client

```python
import rpyc

conn = rpyc.connect("localhost", 18861)

# RPC for add
x = conn.root.add(4,7)
assert x == 11

# RPC for div
try:
    result = conn.root.div(4, 2)
    print("Division result:", result)
    conn.root.div(4, 0)
except ZeroDivisionError:
    print("Division by zero error")

# RPC for fib
print(conn.root.fib(1000))
```

On the client side, the remote service is accessed via the root object of the connection (`conn.root`).
This root object is a network reference (`netref`) to the service instance running on the server.
Any operation on `conn.root` is automatically forwarded to the server, making the remote service appear local.
This mechanism allows the client to transparently call exposed methods and access attributes of the server service.

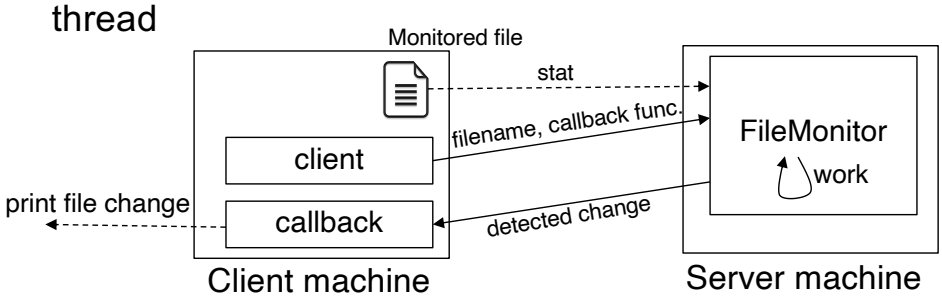
RPyC: async operation

- A key feature of RPyC
  - Client starts remote call and continues without blocking
  - Returns `AsyncResult` object that will *eventually* hold the result
- Asynchronous behavior must be *explicitly* enabled: to make a call asynchronous, wrap it with `async_()`
- Then, client can
  - test `AsyncResult` object for completion using `ready()`
  - wait until completion using `wait()`
  - retrieve result using `value`
  - set timeout for result using `set_expiry()`
  - register a callback function to execute when result arrives using `add_callback()`
- No guaranteed order of execution for async requests

See https://rpyc.readthedocs.io/en/latest/docs/async.html

RPyC: async operation and events

- Events can be implemented as asynchronous callbacks
  - Server produces events, client consumes them asynchronously
- Example: FileMonitor
  - Server periodically monitors file on the client machine using `os.stat()`
  - On file change, server sends an event to client (using an async callback) and provides old and new `stat` results
  - At client side, incoming events are handled by a background thread



RPyC example: FileMonitor server

```python
import rpyc
import os
import time
from threading import Thread

class FileMonitorService(rpyc.Service):
    class exposed_FileMonitor(object): # expose class
        def __init__(self, filename, callback, interval = 1):
            self.filename = filename    # file to monitor
            self.interval = interval    # monitoring interval
            self.last_stat = None
            self.callback = rpyc.async_(callback) # async callback
            self.active = True
            self.thread = Thread(target = self.work)
            self.thread.start()
        def exposed_stop(self): # must be exposed since client calls it remotely
            self.active = False
            self.thread.join()
        def work(self):     # check for file changes
            while self.active:
                stat = os.stat(self.filename) # get status of monitored file
                if self.last_stat is not None and self.last_stat != stat:
                    self.callback(self.last_stat, stat) # notify client of change
                self.last_stat = stat
                time.sleep(self.interval) # monitor file status periodically

if __name__ == "__main__":
    from rpyc.utils.server import ThreadedServer
    # spawn a thread for each connection
    ThreadedServer(FileMonitorService, port = 18871).start()
```

Exposing the class, client can call remotely the class constructor with `conn.root.FileMonitor(...)`, but only the `exposed_` methods are accessible remotely

RPyC example: FileMonitor client

```python
import rpyc
import time
import os

# File to monitor
filename = "/tmp/floop.bloop"
if os.path.exists(filename):
    os.remove(filename)

# Open file to monitor
f = open(filename, "wb", buffering=0)

# Connect to RPyC server that monitors file changes
conn = rpyc.connect("localhost", 18871)

# Start background thread that handles incoming asynchronous events from server
# Necessary for receiving callbacks without blocking the client
bgsrv = rpyc.BgServingThread(conn)

# Define the callback function on client side
# Called by server whenever the monitored file changes
def on_file_changed(oldstat, newstat):
    print("\nfile changed")
    print(f"    old stat: {oldstat}")
    print(f"    new stat: {newstat}")

# Create the file monitor object and pass filename and callback function as arguments
filemon = conn.root.FileMonitor(filename, on_file_changed)

# Wait for file monitor to check the initial file state before changing it
time.sleep(2)
```

Each client gets its own FileMonitor object that runs a thread

RPyC example: FileMonitor client

```
# Change file size and be notified by file monitor about change
print("change file size")
f.write(b"loop")

# Wait for file monitor to have a look at changed file
time.sleep(2)

# Change file size again
print("change size again")
f.write(b"groop")
time.sleep(2)

f.close()
filemon.stop()
bgsrv.stop()    # stop background thread
conn.close()
```


RPyC: summing up key features

- Transparency: transparent access to remote objects and methods
- Only exposed classes, methods, or attributes are visible remotely
- Both synchronous and asynchronous calls
- Allows callbacks: server can call functions defined on client
- Concurrency: multiple clients can interact safely with the same server
- Pure Python, no stubs, no IDL, no service registry
- Failure semantics: best effort, requires explicit failure handling
  - No built-in timeouts or retries, no server deduplication

RPC in Go

- Let's introduce the Go programming language
  http://www.ce.uniroma2.it/courses/sdcc2526/slides/Go.pdf

- What about RPC in Go?

RPC in Go

- Go provides built-in support for RPC
  - Package `net/rpc` https://pkg.go.dev/net/rpc
  - Type-safe RPC mechanism

- Main features
  - Provides remote access to exported methods of an object
  - TCP or HTTP as "transport" protocols
  - Supports concurrent client requests

RPC in Go: defining an RPC service

- Define a type (struct) whose methods will be exposed remotely
- Methods must satisfy the following rules:
  - Method type and method name are exported (start with capital letter)
  - Method has two arguments, both exported or built-in types
    - First argument: input (usually a struct)
    - Second argument: pointer to reply struct for response
  - Method returns an error

Defining an RPC service: example

```
package arith
import "errors"

type Arith struct{}  // Service struct
type Args struct {A, B int}  // Input arguments
type Result int  // Result type for Multiply method
type Quotient struct {Quo, Rem int}  //Result type for Divide method

func (t *Arith) Multiply(args *Args, res *Result) error {
        *res = args.A * args.B
        return nil
}
func (t *Arith) Divide(args *Args, quo *Quotient) error {
        if args.B == 0 {
                return errors.New("divide by zero")
        }
        quo.Quo = args.A / args.B
        quo.Rem = args.A % args.B
        return nil
}
```

RPC in Go: server

- On server side
  - Create a TPC (or HTTP) server to receive incoming requests
  - Use **`Register`** (or **`RegisterName`**) to make an object's exported methods visible as RPC procedures and publish them on server

```
func (server *Server) Register(rcvr any) error
func (server *Server) RegisterName(name string, rcvr
any) error
```

  - Takes interface as input: `any` is an alias for `interface{}`, allows any value (tipically a pointer to a struct)
  - `Register` uses the type name as service name and registers methods
  - `RegisterName` allows to specify a custom service name

RPC in Go: server

  - Use **`Listen`** to create a network listener and announce the server on a specific port

```
func Listen(network, address string) (Listener, error)
```

  - Use **`Accept`** to accept connections on the listener and serve requests

```
func (server *Server) Accept(lis net.Listener)
```

    - Concurrency out of the box: `Accept` spawns a separate goroutine for each request
    - `Accept` blocks; to handle other tasks concurrently in main, call `Accept` in a goroutine
  - Can also use an HTTP handler for RPC messages
    - Use `HandleHTTP` and `http.Serve`
    - See example on course site

RPC in Go: client

- On client side
  - Use **`Dial`** to connect to RPC server at the specified address and port using the given transport protocol (e.g., `"tcp"`)

```
func Dial(network, address string) (*Client, error)
```

    - Use `DialHTTP` for HTTP connection
  - Use **`Call`** to call synchronous RPC
    - `Call` waits for the remote procedure to complete

```
func (client *Client) Call(serviceMethod string, args
any, reply any) error
```

  - Use **`Go`** to call asynchronous RPC
    - `Go` invokes the call asynchronously and signals completion via `Done` channel in `Call` struct

```
func (client *Client) Go(serviceMethod string, args
any, reply any, done chan *Call) *Call
```


RPC in Go: client

- On client side
  - Struct **`Call`** represents an active RPC

```
type Call struct {
    ServiceMethod string  // The name of the service and method
                          // to call.
    Args any // The argument to the function (*struct)
    Reply any // The reply from the function (*struct)
    Error error // After completion, the error status.
    Done chan *Call // Receives *Call when Go is complete.
}
```

RPC in Go example: calculator

- Let's consider a simple RPC calculator with two functions: multiply and divide two integers
- Code available on course site

RPC in Go: synchronous call

- Need some setup in advance of this, see example
- `Call` makes a blocking RPC call: invokes the remote function, waits for it to complete, and returns its error status

```
// Synchronous call
args := arith.Args{n1, n2}
var mulReply arith.Result
err = client.Call("Arithmetic.Multiply", args, &mulReply)
if err != nil {
   log.Fatal("Error in Arithmetic.Multiply:", err)
}
fmt.Printf("Arithmetic.Multiply: %d*%d=%d", args.A, args.B,
   mulReply)
```

```
func (client *Client) Call(serviceMethod string, args any, reply any)
error
```

RPC in Go: asynchronous call

- Go uses a channel as parameter to retrieve the RPC reply when the call is complete
- The Done channel will signal when the call is complete by returning the same `Call` object
  - If no channel in input, a new one will be allocated by Go

```
// Asynchronous call
divReply := new(Quotient)
divCall := client.Go("Arithmetic.Divide", args, divReply, nil)
replyCall = <- divCall.Done
// check errors, print, etc.
```

channel

```
func (client *Client) Go(serviceMethod string, args any, reply any,
done chan *Call) *Call
```

- Go internal implementation: see https://go.dev/src/net/rpc/client.go?s=8029:8135 - L284

Go's RPC semantics

- Client-side: no automatic retries and built-in timeouts
- Client returns an error if no reply is received
  - TCP timeout
  - Server may not have received request
  - Server processed request but a failure occurred before sending reply or a network failure occurred
- Server-side: no automatic deduplication of requests
  - Client request has a unique sequence number but is not used for deduplication on server
  - Sequence number is used on client for request tracking
- Best-effort model
- Developer responsibility to implement retries, timeouts, and server-side deduplication

RPC in Go: marshaling and unmarshaling

- By default, Go uses `encoding/gob` package for marshaling (encoding) and unmarshaling (decoding) RPC parameters https://pkg.go.dev/encoding/gob
  - When you use `net/rpc`, gob is the default encoding/decoding mechanism used automatically
- gob manages binary streams (gobs) exchanged between Encoder (sender) and Decoder (receiver)
  - Each item in the stream is *self-describing*: item is preceded by a specification of its type; pointers are not transmitted, only the values they point to are
- Basic usage: create an encoder, transmit values, use a decoder to receive and decode values
  - Both RPC client and server must be written in Go
- Use `gob` explicitly when you need encoding/decoding of Go data outside the automatic RPC mechanism or for persistence

RPC in Go: marshaling and unmarshaling

- Alternatives to `gob` for Go RPC

1. `net/rpc/jsonrpc` provides JSON-RPC encoding https://pkg.go.dev/net/rpc/jsonrpc
   - Implements JSON-RPC 1.0 for `rpc` package
   - Provides ClientCodec and ServerCodec
   - JSON: human-readable but less efficient than `gob`
2. Protocol buffers and gRPC instead of `net/rpc`

Go RPC: summing up key features

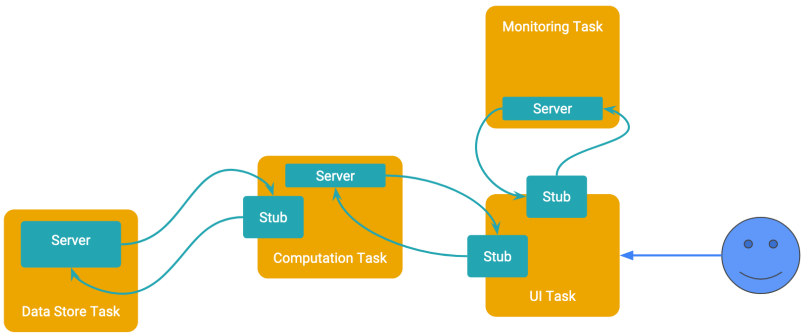
- Transparency: transparent access to remote structs and methods
- Only exposed methods are visible remotely
- Both synchronous and asynchronous calls
- Concurrency: multiple clients can interact with the same server
- No IDL, no service registry
- Go automatically handles marshalling and unmarshalling
- Failure semantics: best effort, requires explicit failure handling
  - No built-in timeouts, retries, server deduplication

Comparing RPC so far

- Let's compare RPC implementations and how they differ in terms of distribution transparency
  - Access transparency
  - Location transparency
  - Concurrency transparency
  - Failure transparency
  - Replication transparency
    - Add server-side load balancer which acts as proxy between RPC clients and replicated RPC servers

Motivation for new RPC middleware

- Large-scale distributed applications composed of **microservices**
  - Microservices architecture: build application as a collection of *independent*, *autonomous* (developed, deployed, and scaled independently), *business capability–oriented*, and *loosely coupled* services
  - Multi-language (i.e., polyglot) development
  - Communication mainly structured as RPCs



gRPC

- High-performance, open-source universal RPC framework https://grpc.io/
- Can run in any environment
  - Multi-language, multi-platform support
- Background and adoption
  - Developed by Google and released in 2015, now CNCF project
  - Widely used by companies like Cisco, Discord, Netflix, Reddit, Spotify, Square
  - Etcd, Kubernetes, containerd, GPC use gRPC for communication between components
  - Has become a cornerstone for cloud-native environments

gRPC

- Main use cases
  - Connect polyglot microservices (request-reply communication style) across and within data centers
    - Supports load balancing, tracing, health checks, authentication
  - Connect devices, mobile apps and browsers to backend services
  - Generate efficient client libraries
- Also core part of many ML systems
  - Gemini, TensorFlow, NVIDIA Triton inference system use it
- And popular for serving models

  https://github.com/tensorflow/serving/blob/master/tensorflow_serving/apis/prediction_service.proto

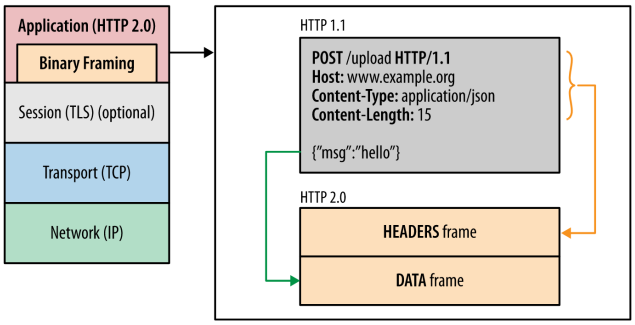
  https://bentoml.com/blog/3-reasons-for-grpc

gRPC: key features

- Transport and communication
  - Uses **HTTP/2** for efficient transport
  - Supports bidirectional streaming, multiplexing, and header compression
- Interface definition and code generation
  - **Protocol Buffers** as IDL
  - Simple service definition, automatic client/server code generation
  - Strict specification prevents errors
- Advanced features
  - Authentication and flow control
  - Blocking or non-blocking client bindings
  - Deadline, timeouts and cancellation
  - Interceptors/middleware: logging, monitoring, retires, metrics

gRPC transport: HTTP/2

- Basic idea: treat RPCs as references to HTTP objects, leveraging HTTP/2 as underlying transport
- Why HTTP/2?
  - Major revision of HTTP, providing significant performance improvements over HTTP 1.x
  - Enables low-latency, high-throughput communication
- HTTP/2 in a nutshell
  - Binary framing layer: requests and responses are divided into small messages framed in binary, making transmission efficient



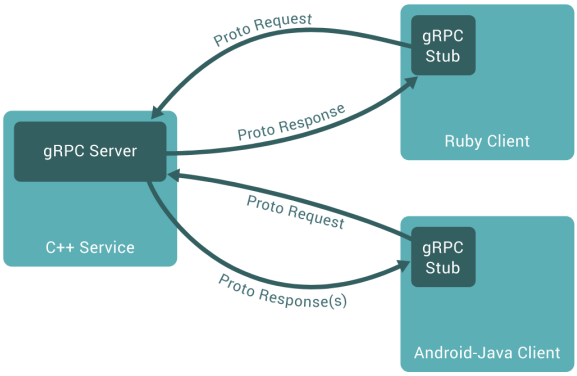
gRPC transport: HTTP/2

- HTTP/2 in a nutshell
  - From messages to streams
    - Stream: bidirectional flow of bytes within a single connection, may carry one or more messages
    - Message: complete sequence of frames, representing a logical request or response
    - Frame: smallest unit of communication, contains a frame header identifying its stream
  - Bidirectional streaming: supports client-server and server-client streaming simultaneously
  - Request/response multiplexing: multiple requests/responses per client share a single connection, allowing efficient use of TCP connections and avoiding head-of-line blocking
  - HTTP header compression: reduces protocol overhead

See https://hpbn.co/http2/

gRPC: Protocol buffers

- gRPC uses Protocol buffers (aka *protobuf*) as:
  - IDL for service interface definition and automatic generation of client stubs and abstract server classes
  - Message interchange format: gRPC messages are serialized using protobufs, thus resulting in compact, efficient message payloads
- Design pattern: proxy pattern
  - Client stub acts as a local proxy to the remote service
  - Server skeleton handles incoming RPC calls



Protocol buffers

- Google's mature open-source mechanism to serialize structured data
- Binary data representation: compact and efficient
- Strongly typed: enforces data safety and prevents erros

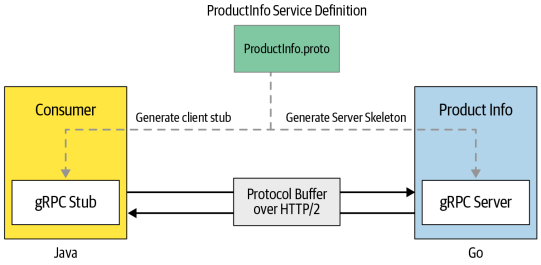
```
syntax = "proto3";

message Person {
  string name = 1;
  int32 id = 2;
  string email = 3;
}
```

- Data types are structured as messages
  - Message: logical record of information containing a series of name-value pairs called fields
  - Each field has a unique number (e.g., `string name = 1`)
    - Field numbers are used in the binary encoding to identify data

Protocol buffers: example

- ProductInfo service interface



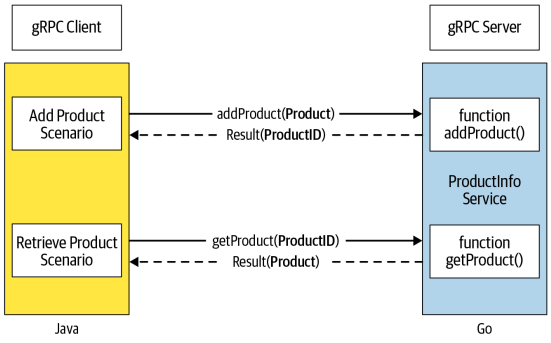

```
// ProductInfo.proto
syntax = "proto3";
package ecommerce;

service ProductInfo {
    rpc addProduct(Product) returns (ProductID);
    rpc getProduct(ProductID) returns (Product);
}

message Product {
    string id = 1;
    string name = 2;
    string description = 3;
}

message ProductID {
    string value = 1;
}
```

Client-server interaction



gRPC: basic steps

1. Define service and messages: write `.proto` file using protobufs as IDL and describing
   - Service: collection of remote methods
   - Message types exchanged between client and server
2. Generate server and client code
   - Use `protoc` (protocol buffer compiler) in your preferred language(s) from proto definition
   - Go: run `protoc` manually to generate stubs
   - Java: use build tools (e.g., Maven, Gradle)
3. Implement service and client
   - Use language-specific gRPC API (e.g., Go, Java, Python) to write client and server applications
   - Let's consider Go: gRPC-Go https://grpc.io/docs/languages/go

gRPC: helloworld example in Go

- See https://grpc.io/docs/languages/go/quickstart/ and https://github.com/grpc/grpc-go/tree/master/examples/helloworld/

1. Define service (`helloworld.proto` file)

```
package helloworld;

// The greeting service definition.
service Greeter {
  // Sends a greeting
  rpc SayHello (HelloRequest) returns (HelloReply) {}
}

// The request message containing the user's name.
message HelloRequest {
  string name = 1;
}

// The response message containing the greetings
message HelloReply {
  string message = 1;
}
```


gRPC: helloworld example in Go

2. Compile gRPC service definition

```
$ protoc --go_out=. --go_opt=paths=source_relative \
    --go-grpc_out=. --go-grpc_opt=paths=source_relative \
    helloworld/helloworld.proto
```

- Automatically generated files:
  - `helloworld.pb.go`
    - Contains Go structs for protobuf message types: message types have become Go structs
    - Provides methods to populate, serialize, and retrieve request and response data
  - `helloworld_grpc.pb.go`
    - contains Go interfaces (*stub*) for RPC calls: RPC definitions have become Go interfaces
    - Client stub interface to call service methods
    - Server interface to implement service methods
    - Uses message structs from `helloworld.pb.go`

gRPC: helloworld example in Go

3. Create gRPC server: composed of two parts
   a. Implement the interface generated from `.proto` service definition: the actual "work"

```
func (s *server) SayHello(ctx context.Context,
   in *pb.HelloRequest) (*pb.HelloReply, error) {
    // Implement service logic here
}
```

   - `ctx`: carries deadlines, cancellation signals and metadata
   - `in`: request message from the client

gRPC: helloworld example in Go

3. Create gRPC server: composed of two parts
   b. Create and run gRPC server: listen on a network port, create a new gRPC server instance, register the service implementation, pass the listener to the gRPC server

```
lis, err := net.Listen("tcp", port)
if err != nil {
    log.Fatalf("failed to listen: %v", err)
}
s := grpc.NewServer()
pb.RegisterGreeterServer(s, &server{})
s.Serve(lis)
```

   - `Serve` blocks and starts handling requests

gRPC: helloworld example in Go

4. Create gRPC client
   - Establish a *gRPC channel* (connection) to communicate with server using `Dial`

```
conn, err := grpc.Dial(address, opts…)
```

     - `opts` can include options like `grpc.WithInsecure()`, TLS credentials, etc.
   - Create a client stub to perform RPCs: get it using `pb.NewGreeterClient` provided by `pb` package (generated from `.proto` file)

```
c := pb.NewGreeterClient(conn)
```

   - Call service methods on client stub: send request, receive response

```
r, err := c.SayHello(ctx, &pb.HelloRequest{Name: name})
```

     - Create and populate a request protobuf (`HelloRequest`)
     - Pass a `context` to support deadlines, cancellation or metadata
     - Receive a response protobuf (`HelloReply`)

gRPC: updating a service

- gRPC simplifies application updates
  - Incremental updates without breaking existing functionalities
- To add a new method
  1. Modify the service definition (`.proto` file): add new methods, update message types
  2. Regenerate client and server stubs
  3. Update server implementation to implement the new method
  4. Update client code to call the new method

gRPC: updating helloworld example

- Let's update helloworld adding `SayHelloAgain()`
  1. Update the `.proto` file
  2. Regenerate gRPC code using `protoc`
  3. Update server implementation with the new method

```
func (s *server) SayHelloAgain(ctx context.Context, in
   *pb.HelloRequest) (*pb.HelloReply, error) {
    log.Printf("Received: %v", in.GetName())
    return &pb.HelloReply{Message: "Hello again " +
   in.GetName()}, nil
}
```

  4. Update client code calling the new method

```
r, err = c.SayHelloAgain(ctx, &pb.HelloRequest{Name: name})
 if err != nil {
    log.Fatalf("could not greet: %v", err)
 }
 log.Printf("Greeting: %s", r.GetMessage())
```


gRPC: ProductInfo example

- ProductInfo microservice which manages products and their information
- Client operations:
  - Add new products, retrieve product details

1. Define service in `.proto` file (see slide 122)
2. Implement server and client in Go
3. Implement server and client in Java

Go code: Teams/course website

Java code: https://github.com/grpc-up-and-running/samples/tree/master/ch02

See chapter 2 of gRPC: Up and Running
https://www.oreilly.com/library/view/grpc-up-and/9781492058328/

gRPC: service method types

- gRPC supports 4 kinds of service methods
  - Defined in `.proto` file
  - Example: route guide
    https://github.com/grpc/grpc-go/tree/master/examples/route_guide

1. Simple (unary) RPC
   - Client sends a single request to server and waits for a single response
   - A normal function call

```
rpc GetFeature(Point) returns (Feature) {}
```

2. Server-streaming RPC
   - Client sends one request to server
   - Server responds with a stream of messages
   - Client reads until the stream ends
   - Message order is guaranteed within an RPC call

```
rpc ListFeatures(Rectangle) returns (stream Feature) {}
```


gRPC: service method types

3. Client-streaming RPC
   - Client sends a stream of messages to the server
   - Server reads all messages and returns a single response
   - gRPC guarantees message ordering within each RPC call

```
rpc RecordRoute(stream Point) returns (RouteSummary) {}
```

4. Bidirectional streaming RPC
   - Both client and server send streams of messages simultaneously (i.e., full duplex)
   - Each side can read and write independently
   - Server can wait until it has received all client's messages before writing its messages, or server and client can play "ping-pong"
   - gRPC preserves message ordering in each stream

```
rpc RouteChat(stream RouteNote) returns (stream
RouteNote) {}
```

gRPC: route guide example in Go

- Goal: demonstrates all four types of gRPC communication in Go using a location-based route guide service
- protobufs: service and message definitions in `routeguide/route_guide.proto`
- Server: implements gRPC service methods (`GetFeature`, `ListFeatures`, `RecordRoute`, `RouteChat`)
- Client: calls RPCs using Go gRPC stubs
- Transport: supports both plaintext and TLS-secured connections

gRPC: distribution transparency

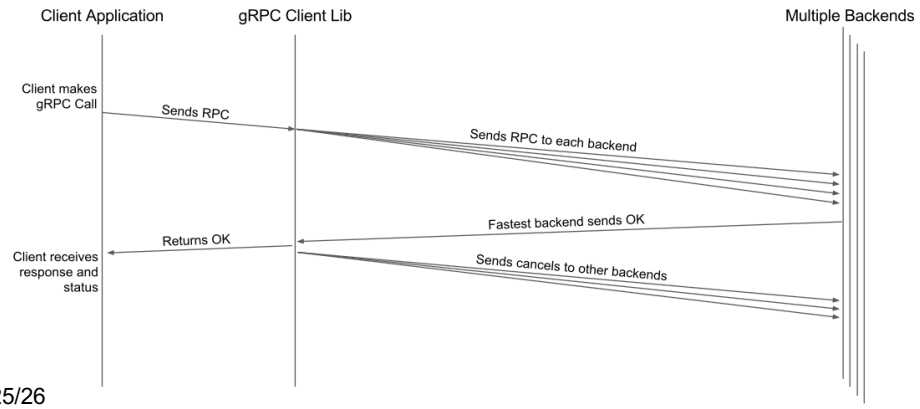
- Access transparency
  - Achieved through the usual proxy pattern
  - Cross-language
- Location transparency
  - Default: DNS name resolver
  - Custom name resolution
    https://grpc.io/docs/guides/custom-name-resolution
  - Service discovery supported via pluggable mechanisms (e.g., Consul, Eureka)
  - Integration with service meshes (e.g., Istio, Consul) that provide service discovery
    - Infrastructure layer for managing service-to-service communication in microservices

gRPC: distribution transparency

- Concurrency transparency
  - Provides concurrency-safe APIs, but concurrency management remains application-dependent (no full transparency)
  - gRPC-Go uses goroutines and synchronization mechanisms https://github.com/grpc/grpc-go/blob/master/Documentation/concurrency.md
- Replication transparency
  - Integrates with client-side or external load balancers (e.g., Envoy) to distribute client requests across replicated servers https://grpc.io/docs/guides/custom-load-balancing
  - Replication logic (e.g., keeping replica consistent) must be handled by the application
- Failure transparency
  - At-most-once semantics by default
  - Some failures (e.g., server crashes mid-call) may result in *partial execution*, not rolled back automatically: no exactly-once semantics

gRPC: distribution transparency

- Failure transparency
  - Retry strategies can be configured for transient failures, including exponential backoff https://grpc.io/docs/guides/retry → retry with backoff pattern
  - Request hedging: send multiple copies of the same request to reduce tail latency https://grpc.io/docs/guides/request-hedging
    - Inspired by: Dean & Barroso, The Tail at Scale, 2013 https://research.google/pubs/the-tail-at-scale/
  - Retries and hedging must be configured, not automatic



gRPC: additional features

- Security: SSL/TLS support
  - How to use
    - Generate certificates
      - Development: use self-signed certificates (via OpenSSL)
      - Production: use valid, trusted certificates from Certificate Authority
    - Server side: configure gRPC server with SSL/TLS certificates and private key
    - Client side: configure gRPC client to trust server's certificates
    - Mutual TLS (mTLS): for client-server authentication, ensuring both sides are verified
    - Example: https://github.com/grpc/grpc-go/tree/master/examples/features/encryption
- Benchmarking and load testing
  - ghz: modern load testing tool for gRPC https://ghz.sh
  - Test different gRPC call types (e.g., unary vs. streaming)
  - Benchmark with different payload sizes and concurrency levels to simulate real-world traffic

gRPC: weaknesses

- Limited browser support
  - Cannot directly call a gRPC service from a browser
  - Requires gRPC-Web for communication (but limited features) https://grpc.io/docs/platforms/web/basics/
- Binary protocol
  - protobufs is efficient but not human-readable
  - Requires additional tools like gRPC command-line tool or Wireshark https://wiki.wireshark.org/gRPC
- Load balancing and traffic management
  - No built-in server-side load balancing
  - Service mesh dependency
- Complexity in large-scale systems

References

- Chapter 4 of van Steen & Tanenbaum book
- Remote Procedure Calls https://www.linuxjournal.com/article/2204
- Java Remote Method Invocation Specification https://docs.oracle.com/en/java/javase/23/docs/specs/rmi/
- Trail: RMI https://docs.oracle.com/javase/tutorial/rmi/
- RPyC tutorial https://rpyc.readthedocs.io/en/latest/tutorial.html
- RPyC documentation https://rpyc.readthedocs.io
- Go's package rpc https://pkg.go.dev/net/rpc
- RPC in the Go standard library, in Building Microservices with Go https://subscription.packtpub.com/book/web-development/9781786468666/1/ch01lvl1sec12/rpc-in-the-go-standard-library
- gRPC - Up and Running https://www.oreilly.com/library/view/grpc-up-and/9781492058328/